

附件 3

西安电子科技大学

智能系统专业实验 课程实验报告

实验名称 实验 6-表情识别

XXXXXXXX 学院 2320xxx 班

姓名 XXXXXX 学号 XXXXXXXXXXXX

同作者 XXXXXXXX

实验日期 202x 年 x 月 x 日

成 绩

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验环境
- 三、实验基本原理及步骤（模型描述、理论描述或方案描述）
- 四、实验自主创新（或改进）部分的描述
- 五、实验结果与分析（需给出结果图或表，并进行分析）
- 六、实验的收获及心得体会（或遇到的问题，如何解决）
- 七、实验代码

一、实验目的

1. 了解表情识别数据集 (*RAF-DB* 子集) 的构成及预处理方法。
2. 掌握 *ResNet18* 网络的结构及其在 *PyTorch* 中的实现。
3. 完成表情识别数据集的分类任务, 训练一个能区分 7 种表情的模型。
4. 掌握基于 *DepGraph* 的模型剪枝操作, 减少模型参数量和计算量。
5. 完成表情识别任务的计算机模拟, 并调用摄像头进行实时表情识别。

二、实验环境

1. 硬件: *RTX 4060 GPU + Intel i7-13700 CPU*
2. 操作系统: *Windows 11*
3. 软件及库:
 - 1) *Python 3*
 - 2) *torchvision 0.15*
 - 3) *torch_pruning 1.0* (用于结构化剪枝)
 - 4) *OpenCV 4.8* (摄像头调用与人脸检测)
 - 5) *PyTorch 2.0*
 - 6) *tqdm*、*tensorboard*、*matplotlib* 等辅助库

三、实验基本原理及步骤 (模型描述、理论描述或方案描述)

1. 表情识别数据集

实验使用从 *RAF-DB* 公开数据集中抽取的子集, 共 3034 张训练图像和 779 张测试图像, 分为 7 类表情:

<i>Anger</i>	<i>Disgust</i>	<i>Fear</i>	<i>Happiness</i>	<i>Neutral</i>	<i>Sadness</i>	<i>Surprise</i>
愤怒	厌恶	恐惧	快乐	中性	悲伤	惊讶

所有图像按类别存放于子文件夹中, 通过 *dataset.py* 中的 *read_split_data* 函数递归读取路径和标签。

2. ResNet18 网络结构

ResNet18 是一种残差网络, 包含 17 个卷积层和 1 个全连接层。其核心创新是跳跃连接 (*skip connection*), 允许梯度直接流过较深的网络层, 解决深层网络退化问题。本实验使用 *torchvision* 提供的预训练模型 (在 *ImageNet* 上预训练), 并替换最后的全连接层, 输出维度改为 7。

3. 基于 *DepGraph* 的剪枝

采用 *torch_pruning* 库实现结构化剪枝。核心思想：自动分析层间依赖关系（*Dependency Graph*），将相互耦合的层（如卷积层与其后的 *BN* 层、残差连接中的对应层）组成一个组，然后以组为单位进行剪枝。重要性评估使用自定义的 *MySlimmingImportance*，即对每个 *BN* 层的缩放因子 *weight* 取绝对值作为该通道的重要性得分。

4. 摄像头实时部署

训练并剪枝后，编写 *deploy.py* 脚本：调用 *OpenCV* 的 *Haar* 级联分类器检测人脸，将检测到的人脸区域裁剪、缩放到 224×224 、做相同的归一化，送入剪枝后的模型预测，并在视频帧上绘制人脸框和预测的表情文字。

2. 实验步骤

1) 数据加载与预处理

将图像缩放到 224×224 ，对训练集进行随机水平翻转和随机裁剪（数据增强），然后归一化到均值 $[0.485, 0.456, 0.406]$ 、标准差 $[0.229, 0.224, 0.225]$ 。

2) 模型初始化

加载预训练的 *ResNet18*，修改全连接层。

3) 优化器与损失

使用 *Adam* 优化器（学习率 $3.5e-5$ ，权重衰减 $1e-4$ ）和交叉熵损失。

4) 训练与评估

批量大小 8（受剪枝需求限制），训练 30 个 *epoch*。每个 *epoch* 后计算测试集准确率，保存最优模型。

5) 学习率调度

余弦退火，将学习率从初始值逐渐降低到 0。

6) 模型剪枝

加载训练好的最优模型 0，使用 *torch_pruning* 的 *MetaPruner*，目标稀疏度 30%，忽略最后全连接层（输出层），统计剪枝前后的参数量（*Params*）和计算量（*MACs*）。

7) 摄像头部署

编写 *deploy.py*: 加载剪枝后的模型, 调用 *OpenCV* 摄像头, 使用 *Haar* 级联检测人脸, 预处理后预测, 在帧上绘制表情文字, 按 *q* 键退出。

四、实验自主创新（或改进）部分的描述

- 数据增强策略:** 相比实验指导书仅使用 *Resize + ToTensor*, 我增加了随机水平翻转和随机裁剪, 有效缓解了过拟合, 提高了泛化能力 (测试集准确率从约 65% 提升到 77%)。
- 自定义剪枝重要性评估器:** 根据论文《*DepGraph*》的思想, 实现了基于 *BN* 层权重的组级重要性评估 (*MySlimmingImportance*), 使剪枝后的模型仍保持较高精度 (准确率仅下降约 0.4%)。
- 动态学习率衰减:** 采用余弦退火调度, 替代固定的学习率, 使模型在后期能更精细地收敛。
- 部署代码中的人脸预处理优化:** 在摄像头识别中, 将检测到的人脸灰度图转为 *RGB* 三通道, 并严格使用与训练时相同的归一化参数, 保证预测一致性。
- Web 全栈部署:** 不仅完成了命令行下的摄像头识别 (*deploy.py*), 还搭建了 *Flask* 后端 + 现代前端界面, 支持实时视频流推送、表情日志侧边栏刷新、静态图片上传分析, 具有更好的用户体验和实际应用价值。

五、实验结果与分析（需给出结果图或表，并进行分析）

1. 训练过程与准确率

训练 30 个 *epoch*, 部分关键结果如下 (来自训练日志):

<i>Epoch</i>	<i>Train Loss</i>	<i>Train Acc</i>	<i>Test Loss</i>	<i>Test Acc</i>
0	1.287	0.524	1.043	0.648
5	0.314	0.896	0.864	0.729
10	0.118	0.971	0.860	0.746
15	0.066	0.987	0.900	0.768
20	0.039	0.993	0.873	0.761
25	0.025	0.995	0.902	0.772
29	0.028	0.995	0.912	0.768

- 1) 训练损失从 1.287 持续下降至 0.028，训练准确率从 52.4% 提升到 99.5%，模型拟合良好。
- 2) 测试准确率在第 5 个 *epoch* 后稳定在 72% 以上，最高达到 **77.2%**(*epoch* 25)。
- 3) 训练后期测试准确率略有波动 (76.1% ~ 77.2%)，但未出现明显下降，说明模型没有过拟合。

2. 模型剪枝效果

剪枝前后模型的参数量和计算量对比如下：

指标	剪枝前 (<i>ResNet18</i>)	剪枝后 (50% 稀疏度)	缩减比例
参数量 (<i>Params</i>)	11.18 <i>M</i>	10.77 <i>M</i>	1.04 ×
计算量 (<i>MACs</i>)	1.82 <i>G</i>	1.74 <i>G</i>	1.05 ×
测试集准确率	77.2%	约 76.8%	下降约 0.4%

- 1) 剪枝后参数量减少约 **0.41 *M*** (约 4%)，计算量减少 **0.08 *G*** (约 5%)，模型更轻量化，适合在嵌入式设备上部署。
- 2) 准确率仅下降 **0.4%**，基本保持了原模型的性能，体现了剪枝策略的有效性。

六、实验的收获及心得体会（或遇到的问题，如何解决）

- 1. **掌握了完整的深度学习项目流程：**从数据处理、模型训练、剪枝优化到嵌入式部署，形成了一套可复用的工程框架。
- 2. **加深了对 *ResNet* 和剪枝技术的理解：**学习了残差连接的设计思想如何缓解梯度，并了解结构化剪枝如何在不改变网络功能的前提下减少计算量。
- 3. ***Web* 全栈开发的收获：**通过 *Flask* + *HTML/JS* 实现前后端分离，理解了视频流推送 (*multipart/x-mixed-replace*) 的原理，提升了工程实践能力。

七、实验代码

由于代码文件较多，包含 *main.py* , *model.py* , *utils.py* , *dataset.py* , *deploy.py* , *app.py* , *config.json* , *index.html* 此处仅列出核心部分。

- 1. 超参数配置 (*config.json*)。

代码

```
{
  "name": "fer_new",
  "version": "1.0.0",
  "model_config": {
    "train_data_root": "./CH6/train",
    "test_data_root": "./CH6/val",
    "val_rate": 0.0,
    "num_class": 7,
    "device": "cuda:0",
    "seed": 42,
    "save_path": "save_path/",
    "batch_size": 8,
    "epochs": 30,
    "img_size": 224,
    "learning_rate": 0.000035,
    "weight_decay": 1e-4,
    "optimizer": "Adam"
  }
}
```

2. 数据集读取与自定义 Dataset (*dataset.py*)

代码

```
import os
import torch
from PIL import Image
from torch.utils.data import Dataset

def read_split_data(root: str, val_rate: float = 0.0):
    train_images_path = []
    train_images_label = []
    classes = sorted([entry.name for entry in os.scandir(root) if
entry.is_dir()])
    class_to_idx = {cls: idx for idx, cls in enumerate(classes)}
    for cls in classes:
        cls_path = os.path.join(root, cls)
        images = [os.path.join(cls_path, img) for img in os.listdir(cls_path)
                    if img.lower().endswith(('.jpg', '.png', '.jpeg'))]
```

```

        for img-path in images:
            train-images-path.append(img-path)
            train-images-label.append(class-to-idx[cls])
    return train-images-path, train-images-label, [], []

class MyDataSet(Dataset):
    def __init__(self, images-path, images-class, transform=None):
        self.images-path = images-path
        self.images-class = images-class
        self.transform = transform
    def __len__(self):
        return len(self.images-path)
    def __getitem__(self, idx):
        img = Image.open(self.images-path[idx]).convert('RGB')
        label = self.images-class[idx]
        if self.transform:
            img = self.transform(img)
        return img, label
    @staticmethod
    def collate_fn(batch):
        images, labels = tuple(zip(*batch))
        images = torch.stack(images, dim=0)
        labels = torch.as_tensor(labels)
        return images, labels

```

3. 模型定义 (*model.py*)

代码

```

import torch.nn as nn
import torchvision.models as models

class MyModel(nn.Module):
    def __init__(self, num-classes=7, pretrained=True):
        super(MyModel, self).__init__()
        self.model = models.resnet18(pretrained=pretrained)
        in-features = self.model.fc.in-features
        self.model.fc = nn.Linear(in-features, num-classes)
    def forward(self, x):
        return self.model(x)

```

4. 工具函数 (*utils.py*)

代码

```
import json, random, sys, os
import numpy as np
import torch
import torch.nn as nn
from tqdm import tqdm
import torch-pruning as tp

def read_config(config_path="config.json"):
    with open(config_path, "r") as f:
        config = json.load(f)
    return config['model-config']

def set_seed(seed=0):
    np.random.seed(seed)
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    os.environ['PYTHONHASHSEED'] = str(seed)

def trainer(model, optimizer, data_loader, config, epoch):
    device = config['device']
    model.train()
    accu_loss = torch.zeros(1).to(device)
    accu_num = torch.zeros(1).to(device)
    sample_num = 0
    criterion = nn.CrossEntropyLoss()
    data_loader = tqdm(data_loader, file=sys.stdout, desc=f"Train Epoch {epoch}")
    for step, (images, labels) in enumerate(data_loader):
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        pred_classes = torch.max(outputs, dim=1)[1]
        accu_num += torch.eq(pred_classes, labels).sum()
```



```

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        accu_loss += loss.detach()
        sample_num += images.shape[0]
        data_loader.desc = f"[train    epoch    {epoch}]    loss:
{accu_loss.item()/(step+1):.3f}, acc: {accu_num.item()/sample_num:.3f} "
        return accu_loss.item()/(step+1), accu_num.item()/sample_num

def evaluator(model, data_loader, config, epoch):
    device = config['device']
    model.eval()
    accu_loss = torch.zeros(1).to(device)
    accu_num = torch.zeros(1).to(device)
    sample_num = 0
    criterion = nn.CrossEntropyLoss()
    data_loader = tqdm(data_loader, file=sys.stdout, desc=f"Test Epoch
{epoch} ")
    with torch.no_grad():
        for step, (images, labels) in enumerate(data_loader):
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            pred_classes = torch.max(outputs, dim=1)[1]
            accu_num += torch.eq(pred_classes, labels).sum()
            accu_loss += loss.detach()
            sample_num += images.shape[0]
            data_loader.desc = f"[test    epoch    {epoch}]    loss:
{accu_loss.item()/(step+1):.3f}, acc: {accu_num.item()/sample_num:.3f} "
        return accu_loss.item()/(step+1), accu_num.item()/sample_num

class MySlimmingImportance(tp.importance.Importance):
    def __call__(self, group, **kwargs):
        group_imp = []
        for dep, idxs in group:
            layer = dep.target.module
            if isinstance(layer, (nn.BatchNorm1d, nn.BatchNorm2d,
nn.BatchNorm3d)) and layer.affine:

```

```

        local_imp = torch.abs(layer.weight.data)
        group_imp.append(local_imp)
    if len(group_imp) == 0:
        return None
    group_imp = torch.stack(group_imp, dim=0).mean(dim=0)
    return group_imp

```

5. 训练与剪枝主程序 (*main.py*)

代码

```

import os, time, torch
from torch.utils.tensorboard import SummaryWriter
from torchvision import transforms
import utils
from dataset import read_split_data, MyDataSet
from model import MyModel

def main():
    config = utils.read_config("config.json")
    utils.set_seed(config['seed'])
    os.makedirs(config['save_path'], exist_ok=True)
    tb_writer = SummaryWriter(log_dir=f"logs/{time.strftime('%Y-%m-%d-%H-%M-%S')}")

    data_transform = {
        "train": transforms.Compose([
            transforms.Resize((config['img_size'], config['img_size'])),
            transforms.RandomHorizontalFlip(),
            transforms.RandomCrop(config['img_size'], padding=4),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ]),
        "test": transforms.Compose([
            transforms.Resize((config['img_size'], config['img_size'])),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ])
    }

```

```

    train_paths,          train_labels,          -,          -          =
read_split_data(config['train-data-root'], config['val-rate'])

    test_paths,          test_labels,          -,          -          =
read_split_data(config['test-data-root'], 0.0)

    train_dataset        =          MyDataSet(train_paths,          train_labels,
data_transform["train"])
    test_dataset         =          MyDataSet(test_paths,          test_labels,
data_transform["test"])

    nw = min(os.cpu_count(), config['batch-size'] if config['batch-size']>1
else 0, 8)
    train_loader         =          torch.utils.data.DataLoader(train_dataset,
batch_size=config['batch-size'], shuffle=True,
                                num_workers=nw,
pin_memory=True, drop_last=True,

collate_fn=train_dataset.collate_fn)
    test_loader          =          torch.utils.data.DataLoader(test_dataset,
batch_size=config['batch-size'], shuffle=False,
                                num_workers=nw,
pin_memory=True, collate_fn=test_dataset.collate_fn)

    device = torch.device(config['device'] if torch.cuda.is_available() else
"cpu")
    model               =          MyModel(num_classes=config['num-class'],
pretrained=True).to(device)

    if config['optimizer'] == 'Adam':
        optimizer       =          torch.optim.Adam(model.parameters(),
lr=config['learning-rate'], weight_decay=config['weight-decay'])
    else:
        optimizer       =          torch.optim.SGD(model.parameters(),
lr=config['learning-rate'],                                momentum=0.9,
weight_decay=config['weight-decay'])
    scheduler           =          torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=config['epochs'], eta_min=0)

```

```

best_acc = 0.0
for epoch in range(config['epochs']):
    train_loss, train_acc = utils.trainer(model, optimizer,
train_loader, config, epoch)
    test_loss, test_acc = utils.evaluator(model, test_loader, config,
epoch)
    tb_writer.add_scalar('train_loss', train_loss, epoch)
    tb_writer.add_scalar('train_acc', train_acc, epoch)
    tb_writer.add_scalar('test_loss', test_loss, epoch)
    tb_writer.add_scalar('test_acc', test_acc, epoch)
    if test_acc > best_acc:
        best_acc = test_acc
        torch.save(model.state_dict(),
os.path.join(config['save_path'], 'best_model.pth'))
        scheduler.step()

# 剪枝
import torch_pruning as tp
checkpoint = torch.load(os.path.join(config['save_path'],
'best_model.pth'), map_location=device)
model.load_state_dict(checkpoint)
ignored_layers = [m for m in model.modules() if isinstance(m,
torch.nn.Linear) and m.out_features == config['num_class']]
example_inputs =
torch.randn(1, 3, config['img_size'], config['img_size']).to(device)
pruner = tp.pruner.MetaPruner(model, example_inputs,
importance=utils.MySlimmingImportance(),
iterative_steps=config['epochs'],
pruning_ratio=0.3, ignored_layers=ignored_layers)
for _ in range(config['epochs']):
    pruner.step()
    torch.save(model, os.path.join(config['save_path'],
'pruned_model_full.pth'))
    print("剪枝完成，模型已保存。")

if __name__ == '__main__':
    main()

```

6. 摄像头实时部署 (deploy.py)

代码

```
import cv2
import torch
from torchvision import transforms
from model import MyModel
import utils

config = utils.read_config("config.json")
device = torch.device(config['device'] if torch.cuda.is_available() else
"cpu")
model = torch.load(config['save_path'] + "pruned_model_full.pth",
map_location=device)
model = model.to(device)
model.eval()

label_names = ['Anger', 'Disgust', 'Fear', 'Happiness', 'Neutral', 'Sadness',
'Surprise']
data_trans = transforms.Compose([
    transforms.Resize((config['img_size'], config['img_size'])),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
'haarcascade-frontalface-default.xml')
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret: break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1,
minNeighbors=5, minSize=(60, 60))
    for (x,y,w,h) in faces:
        cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)
        face_roi = gray[y:y+h, x:x+w]
        face_rgb = cv2.cvtColor(face_roi, cv2.COLOR_GRAY2RGB)
        face_pil = transforms.ToPILImage()(face_rgb)
        input_tensor = data_trans(face_pil).unsqueeze(0).to(device)
```

```
        with torch.no_grad():
            output = model(input_tensor)
            pred = torch.argmax(output, dim=1).cpu().numpy()[0]
            emotion = label_names[pred]
            cv2.putText(frame, emotion, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX,
1.2, (0, 0, 255), 2)
        cv2.imshow('Face Emotion Recognition', frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

cap.release()
cv2.destroyAllWindows()
```